# Roadmap

| to Advanced | | | | |
|---|---|---|---|---|
| c, module by module. | | | | |
| | | | | |
| | Level | Module | Topic | Description |
| | Beginner | Getting Started | What is Python | High-level, interpreted, general-purpose programming language k |
| | Beginner | Getting Started | Installing Python & IDEs | Setting up Python interpreter, pip, and editors like VS Code / PyCl |
| | Beginner | Getting Started | Running Python Code | Using the REPL, scripts (.py files), and Jupyter notebooks |
| | Beginner | Basics | Variables & Assignment | Naming rules, dynamic typing, multiple assignment |
| | Beginner | Basics | Data Types | int, float, complex, bool, str, NoneType |
| | Beginner | Basics | Type Casting | Converting between types with int(), float(), str(), bool() |
| | Beginner | Basics | Operators | Arithmetic, comparison, logical, assignment, bitwise, identity, mem |
| | Beginner | Basics | Input & Output | input() function and print() formatting, f-strings |
| | Beginner | Basics | Comments & Docstrings | Single-line, multi-line comments and documentation strings |
| | Beginner | Control Flow | if / elif / else | Conditional branching logic |
| | Beginner | Control Flow | for Loops | Iterating over sequences with range() and iterables |
| | Beginner | Control Flow | while Loops | Condition-based repetition |
| | Beginner | Control Flow | break / continue / pass | Loop control statements |
| | Beginner | Data Structures | Strings | Indexing, slicing, string methods, formatting |
| | Beginner | Data Structures | Lists | Creation, indexing, slicing, list methods, mutability |
| | Beginner | Data Structures | Tuples | Immutable sequences, packing/unpacking |
| | Beginner | Data Structures | Sets | Unique collections, set operations (union, intersection) |
| | Beginner | Data Structures | Dictionaries | Key-value pairs, dict methods, iteration |
| | Beginner | Data Structures | Comprehensions | List, dict, and set comprehensions |
| | Beginner | Functions | Defining Functions | def keyword, parameters, return values |
| | Beginner | Functions | Default & Keyword Args | Default parameter values, *args and **kwargs |
| | Beginner | Functions | Lambda Functions | Anonymous single-expression functions |
| | Beginner | Functions | Scope & Namespaces | Local, global, nonlocal, LEGB rule |
| | Beginner | Functions | Recursion | Functions calling themselves, base cases |
| | Intermediate | OOP | Classes & Objects | Defining classes, instantiating objects, attributes |
| | Intermediate | OOP | Constructors | __init__ method and self parameter |
| | Intermediate | OOP | Inheritance | Single, multiple, and multilevel inheritance |
| | Intermediate | OOP | Polymorphism | Method overriding and duck typing |
| | Intermediate | OOP | Encapsulation | Public, protected, private attributes via naming conventions |
| | Intermediate | OOP | Abstraction | Abstract base classes with the abc module |
| | Intermediate | OOP | Magic/Dunder Methods | __str__, __repr__, __len__, __eq__, operator overloading |
| | Intermediate | OOP | Class & Static Methods | @classmethod and @staticmethod decorators |

| | | | | |
|---|---|---|---|---|
| | Intermediate | OOP | Properties | @property decorator for getters/setters |
| | Intermediate | Error Handling | try / except / finally | Catching and handling exceptions |
| | Intermediate | Error Handling | Raising Exceptions | raise keyword and built-in exception types |
| | Intermediate | Error Handling | Custom Exceptions | Creating user-defined exception classes |
| | Intermediate | File Handling | Reading & Writing Files | open(), read(), write(), with statement |
| | Intermediate | File Handling | Working with CSV | csv module for reading/writing CSV files |
| | Intermediate | File Handling | Working with JSON | json module for serialization/deserialization |
| | Intermediate | Modules | Importing Modules | import statement, from...import, aliasing |
| | Intermediate | Modules | Creating Modules & Packages | Organizing code into modules and packages, __init__.py |
| | Intermediate | Modules | pip & Virtual Environments | Installing packages, venv, requirements.txt |
| | Intermediate | Iterators & Generators | Iterators | iter() and next(), the iterator protocol |
| | Intermediate | Iterators & Generators | Generators | yield keyword, generator expressions |
| | Intermediate | Functional Programming | map, filter, reduce | Applying functions across iterables |
| | Intermediate | Functional Programming | Decorators | Function decorators, wrapping behavior with @ |
| | Intermediate | Functional Programming | Context Managers | with statement, __enter__/__exit__, contextlib |
| | Intermediate | Regex | Regular Expressions | re module, pattern matching, search, findall, sub |
| | Intermediate | Testing | Unit Testing | unittest and pytest frameworks, assertions |
| | Intermediate | Testing | Debugging | pdb debugger, logging module |
| | Advanced | Concurrency | Multithreading | threading module, Global Interpreter Lock (GIL) |
| | Advanced | Concurrency | Multiprocessing | multiprocessing module for CPU-bound tasks |
| | Advanced | Concurrency | Asyncio | async/await, event loop, coroutines |
| | Advanced | Advanced OOP | Metaclasses | Customizing class creation with type and metaclasses |
| | Advanced | Advanced OOP | Abstract Base Classes | Interfaces and enforced method contracts |
| | Advanced | Advanced OOP | Descriptors | __get__, __set__, __delete__ protocol |
| | Advanced | Type System | Type Hints | typing module, static type checking with mypy |
| | Advanced | Databases | SQLite with sqlite3 | Connecting, querying, and managing local databases |
| | Advanced | Databases | ORMs (SQLAlchemy) | Object-relational mapping basics |
| | Advanced | Web Development | Flask Basics | Routes, templates, request handling |
| | Advanced | Web Development | Django Basics | Models, views, templates (MVT architecture) |
| | Advanced | Web Development | REST APIs | Building and consuming APIs with requests/Flask |
| | Advanced | Data Science | NumPy | Arrays, vectorized operations, broadcasting |
| | Advanced | Data Science | Pandas | DataFrames, Series, data cleaning and analysis |
| | Advanced | Data Science | Matplotlib/Seaborn | Data visualization basics |
| | Advanced | Best Practices | PEP 8 & Code Style | Python style guide and linting tools |
| | Advanced | Best Practices | Packaging & Distribution | setup.py, pyproject.toml, publishing to PyPI |
| | Advanced | Best Practices | Performance Optimization | Profiling, caching, algorithmic efficiency |
| | | | | |
| | | | | Total Estimated Hours: |

# Cheat Sheet

| thon Syntax Cheat Sheet | | | |
|---|---|---|---|
| ick-reference syntax examples with explanations. | | | |
| | | | |
| tegory | Concept | Syntax / Example | Explanation |
| sics | Variable Assignment | x = 10 | Assigns the integer value 10 to variable x |
| sics | Print Output | print(f"Value: {x}") | f-strings embed expressions inside string literals |
| sics | Multiple Assignment | a, b, c = 1, 2, 3 | Assigns multiple variables in a single line |
| sics | Type Checking | type(x) | Returns the data type of a variable |
| ta Types | String | s = "hello" | Sequence of Unicode characters, immutable |
| ta Types | List | lst = [1, 2, 3] | Ordered, mutable collection of items |
| ta Types | Tuple | t = (1, 2, 3) | Ordered, immutable collection of items |
| ta Types | Set | s = {1, 2, 3} | Unordered collection of unique items |
| ta Types | Dictionary | d = {"key": "value"} | Unordered collection of key-value pairs |
| ntrol Flow | If Statement | if x > 0:<br>  print("positive") | Executes block only if condition is True |
| ntrol Flow | For Loop | for i in range(5):<br>  print(i) | Iterates over a sequence of numbers 0-4 |
| ntrol Flow | While Loop | while x > 0:<br>  x -= 1 | Repeats while condition remains True |
| ntrol Flow | List Comprehension | squares = [i**2 for i in range(10)] | Builds a new list from an expression and loop |
| nctions | Function Definition | def add(a, b):<br>  return a + b | Defines a reusable block of code |
| nctions | Default Argument | def greet(name="World"):<br>  return f"Hi {name}" | Provides fallback value if argument is omitted |
| nctions | Args & Kwargs | def f(*args, **kwargs): pass | Accepts variable number of positional/keyword args |
| nctions | Lambda | square = lambda x: x**2 | Creates a small anonymous function |
| OP | Class Definition | class Dog:<br>  def __init__(self, name):<br>  self.name = name | Defines a blueprint for creating objects |
| OP | Inheritance | class Puppy(Dog):<br>  pass | Puppy inherits attributes/methods from Dog |
| OP | Method Override | def __str__(self):<br>  return self.name | Customizes string representation of an object |
| OP | Static Method | @staticmethod<br>def util(): pass | Method that doesn't access instance or class state |
| ror Handling | Try/Except | try:<br>  risky()<br>except ValueError as e:<br>  print(e) | Catches and handles a specific exception type |
| ror Handling | Finally | finally:<br>  cleanup() | Runs regardless of whether an exception occurred |

| | | | |
|---|---|---|---|
| ror Handling | Custom Exception | class MyError(Exception):<br>  pass | Defines a user-created exception class |
| e Handling | Read File | with open("f.txt") as f:<br>  data = f.read() | Opens and reads a file, auto-closes after block |
| e Handling | Write File | with open("f.txt", "w") as f:<br>  f.write("hi") | Opens a file in write mode and writes text |
| odules | Import | import math<br>math.sqrt(9) | Imports a standard library module |
| odules | Import From | from datetime import datetime | Imports a specific object from a module |
| rators | Generator | def gen():<br>  yield 1<br>  yield 2 | Creates a lazy iterator that yields values one at a ti |
| nctional | Map | list(map(str, [1,2,3])) | Applies a function to every item in an iterable |
| nctional | Filter | list(filter(lambda x: x>0, nums)) | Keeps items where the function returns True |
| nctional | Decorator | @my_decorator<br>def func(): pass | Wraps a function to extend its behavior |
| egex | Search Pattern | re.search(r"\d+", text) | Searches for the first match of a pattern in text |
| egex | Find All | re.findall(r"[a-z]+", text) | Returns all non-overlapping matches as a list |
| ncurrency | Thread | threading.Thread(target=func).start() | Runs a function in a separate thread |
| ncurrency | Async Function | async def fetch():<br>  await something() | Defines a coroutine that can be awaited |
| pe Hints | Function Annotation | def add(a: int, b: int) -> int: | Documents expected argument and return types |
| ata Science | NumPy Array | import numpy as np<br>arr = np.array([1,2,3]) | Creates a fast, fixed-type numerical array |
| ata Science | Pandas DataFrame | import pandas as pd<br>df = pd.read_csv("data.csv") | Loads tabular data into a DataFrame for analysis |
| sting | Assert Equal | assert add(2,3) == 5 | Basic assertion to validate expected behavior |
| sting | Pytest Test | def test_add():<br>  assert add(1,1)==2 | Pytest auto-discovers functions prefixed with test_ |